

Cost-Aware Repository Change Localization with Sparse Impact Planning and Bounded Verification

MSc Research Proposal (V1, supervisor-ready draft)

Ahmed Ehab

September 16, 2026

Abstract

Large language models (LLMs) that edit a software repository must first decide which files a requested change affects. This decision is costly: an explicit file-by-file plan over a repository-scale candidate universe spends hundreds of serialized records per call, and a second-stage reasoner that revisits the scope can multiply inference cost by orders of magnitude. This thesis investigates a representation of impact decisions that is cheap by construction — *preserve-by-omission* sparse plans, which emit only non-preserve decisions — and asks when additional repository reasoning is worth its inference cost. We frame the problem as a correctness–cost trade-off under limited inference budgets and propose to combine (i) a sparse explicit file-level impact policy, (ii) fair cheap/classical/LLM baselines under a common file-selection protocol, and (iii) a *bounded* second-stage verification mechanism that revisits either the task or individual suspicious candidates under a hard budget. Preliminary controlled and real-commit studies show large serialization-cost reductions and meaningful zero-LLM lexical signal, but also that task-level omission risk is not reliably predictable on current development evidence. The thesis therefore evaluates the representation and the correctness–cost Pareto frontier, and tests a pre-registered contingency — candidate-level bounded verification — rather than requiring a task-level risk scorer to succeed.

1 Background and motivation

Repository-level coding agents and impact planners must select affected files before editing. Two separable problems motivate this thesis. First, *impact inference*: did the model identify the right files? Second, *impact-policy representation*: can a complete file-action policy be expressed without spending output budget on hundreds of repeated *preserve* decisions? Our controlled studies show that an explicit full policy at a 16K token cap serializes on average 144 decision records per candidate universe, while a sparse policy that emits only non-preserve decisions serializes a fraction of that, with a large associated reduction in completion tokens and cost.

The second motivating fact is that second-stage reasoning is expensive. A graph-guided baseline under a common protocol consumed on the order of a million tokens per task on the same held-out tasks where our sparse first-pass stays below a few thousand tokens. If a cheap first pass already localizes the bulk of affected files, then the remaining question is whether a *bounded* second look can recover missed files without paying the full always-on cost.

2 Problem statement

Given a repository revision P (parent commit), a natural-language change request q , and a candidate universe $U(P)$ of production files visible at P , a localization system produces an

affected-file set $\hat{S}(P, q)$. The hidden observed change set $S^*(P)$ is used for evaluation only. The thesis studies correctness—cost trade-offs under explicit budgets:

- **Representation cost.** How do sparse (*preserve-by- omission*) and explicit impact policies compare in output tokens, serialized records, cost, latency, and selection fidelity?
- **Bounded second-stage verification.** Can a second mechanism that revisits either the task (*Route A*) or individual suspicious omitted candidates (*Route B*) recover false negatives more efficiently than always-on or random matched-budget verification?

The observed historical diff is treated as an *observed change-set proxy*, never as semantic ground truth.

3 Gap from existing work

Existing lines — agentic graph-guided localization (LocAgent, GraphLocator), repository memory / retrieval-conditioned generation (RepoCoder, RepoAgent), explicit edit planning (CodePlan), agentless localize-then-repair (Agentless), and classical change-impact/propagation analysis — either target semantic fidelity without a cost discipline, or address cheap lexical localization without a second-stage omission-recovery mechanism. None combines an explicit sparse impact-policy representation, a fair cheap/classical/LLM comparison under a common file-selection protocol, and a budgeted omission-aware verifier. Our contribution is not any single component but the combination and its honest, cost-aware evaluation on real commits.

4 Research objectives

1. Build reproducible real-commit localization datasets and protocols (djangoCMS V2-LARGE with an untouched internal test; Saleor as a second real system; NestJS as a cross-ecosystem check).
2. Quantify the representation cost of sparse explicit impact policies and their file-selection fidelity under shared protocols.
3. Evaluate cheap, classical, and LLM localization under common file-level metrics and explicit budgets.
4. Design and test a bounded second-stage verification mechanism (Route A task-level routing, or Route B candidate-level verification), pre-registered and evaluated under matched budgets.
5. Produce correctness—cost Pareto evidence per repository, with macro-averaged cross-repository summaries and a repository-identity confound check.

5 Research questions

- RQ1.** How does sparse explicit impact-policy representation affect inference/output cost and file-selection fidelity?
- RQ2.** How accurately can cheap lexical/static methods localize impacted files relative to LLM planners under shared protocols?
- RQ3.** Can bounded second-stage verification recover missed impacted files more efficiently than always-on or random matched-budget verification?

RQ4. How do these correctness–cost trade-offs transfer across repositories and ecosystems?

If task-level risk modeling remains unsupported by evidence, RQ3 is answered by the candidate-level (Route B) mechanism; *RiskScorer success is not a thesis requirement*.

6 Current preliminary evidence

All numbers are from our audited studies (this repository); they are development or controlled evidence and are reported with their bounds.

- **Preserve-by-omission representation effect (controlled).** In a 16K-cap controlled study (M1B, 60 cells, 6 curated tasks), Sparse-v2 reduced mean completion tokens by $\sim 90\%$, mean serialized decision records by $\sim 95\%$ (corrected metric), recorded API cost by $\sim 82\%$, and latency by $\sim 63\%$ relative to an explicit Full-v2 policy, with both arms fully valid. No universal semantic-superiority claim is made.
- **Real-commit P1** (60 cells; 10 held-out real djangoCMS commits; 2 arms, 3 nested repetitions): semantic selection is difficult — FNR is high; there is *no* Full-vs-Sparse superiority claim (paired deltas’ confidence intervals cross zero). Sparse execution is substantially cheaper.
- **LocAgent (P5):** system-level shared-protocol context only — 402 calls / 32.8M tokens / ~ 9.93 USD estimated on 10 tasks with a 50% empty localization rate; not a reproduction of the published fine-tuned result.
- **Cheap baselines (Protocol A):** BM25 provides a meaningful zero-LLM localization signal on development data; $\text{graphK} \approx \text{path_tokenK}$ with lexical seeds; K is an operating-point curve, not a fixed configuration.
- **Sparse-v2 development inference (90 cells, TRAIN/VALIDATION):** live API, 90/90 valid, 490,747 tokens / \$ 0.184 recorded cost; Sparse omission prevalence 26/30. A task-level RiskScorer is *not* justified on this evidence (4 negatives < 10; no reliable feature above the random band).
- **V2 development inference (2026-09-16):** 431/450 cells within the authorized ceilings (fail-closed token-ceiling stop at 2.5M tokens, \$ 0.873); 144 new V2 development tasks; the v1 candidate omission signal does not replicate within V2 (apparent pooled signal is a candidate-universe size artifact). No multivariable RiskScorer is fitted.
- **Classical/static CIA baseline:** implemented on 150 V2 development cases (zero LLM); BM25 and graph-ranked seeds are the strongest arms; reverse-dependency closure arms are weaker.

These results *motivate* a larger development dataset (V2-LARGE), protection of an untouched internal test, and the bounded-verification design; they are not rewritten as a success.

7 Proposed architecture

1. **Sparse first-pass impact plan:** explicit file-level policy with preserve-by-omission serialization; cheap deterministic and LLM planner variants under the same contract.
2. **Omission-risk estimation (contingency):** only if a larger development set establishes a reliable, stable signal; a simple rule / logistic model, no neural model, no broad feature search.

3. Second-stage verification:

- *Route A*: task-level selective routing (escalate the whole task when risk is high).
- *Route B*: candidate-level bounded verification — identify suspicious omitted candidates using independent structural/history/ retrieval evidence, verify only those under a hard budget, compare with always-verify and random matched-budget verification.

8 Dataset methodology

- **Real history, parent-only inputs.** Inference uses the parent commit P ; the target T is hidden. Live repository HEAD is never used (it moves and can leak future state).
- **Observed change-set proxy.** $P \rightarrow T$ changed production files are an observed proxy, never semantic gold.
- **V2-LARGE (Stage 1).** djangoCMS eligible pool (329 independent candidates after frozen filters/deduplication; 289 untouched beyond the exposed v1 40) split into DEV, an untouched internal test, and a reserve. The split is metadata-only and frozen before any model result.
- **Saleor (Stage 2) and NestJS (Stage 3):** suitability audited (SUITABLE-WITH-DEVIATIONS); protocols frozen; inference is deferred until their sampling frames are reconstructed and splits frozen.

9 Baselines

Random/cheap control, BM25, adaptive cheap retrieval (if justified), classical/static CIA (dependency-propagation closure), Sparse planner, Full planner, faithful LocAgent/reference agent (when reproducible), Sparse + Always Verify, Sparse + Random Verify (matched budget), Sparse + Selective/ Bounded Verify, and an Oracle-routing upper bound. If task-level routing is unsupported, the selective arm becomes candidate-level bounded verification.

10 Evaluation dimensions

Precision, recall, F1, FNR, FN recovery; selected-candidate count; escalation/verification rate; calls; tokens; cost; latency; failures; graph/index build cost separated from query cost; and the correctness–cost Pareto frontier. Per-repository metrics are primary; macro-averages across repositories are reported; pooled metrics are secondary; a repository-identity confound check is included.

11 Statistical plan

- Task is the independent unit; nested repetitions are never treated as independent observations.
- Pre-registered class-balance gate: risk modeling requires at least 10 independent positives and 10 negatives, a predeclared feature above the random band, and direction stability across development/validation.
- Bootstrap confidence intervals over tasks; random-risk bands for AUROC at the observed n ; multiple-comparison awareness (features above the band are compared with the chance expectation).

- Sample-size planning uses development evidence only (never test set); the number of tasks is chosen from the frozen sampling frame, not a round number.

12 Cross-repository generalization plan

Stage 1 djangoCMS V2-LARGE (within-repo large evidence + untouched internal test) → Stage 2 Saleor (different real system, same Python/Django stack) → Stage 3 NestJS (cross-language/cross-framework evidence). Each stage states its claim boundary: within-repo development + internal confirmation; cross-repository confirmation; cross-ecosystem evidence. NestJS is preferred but not forced; a predeclared eligible-pool yield criterion and a JabRef backup are frozen.

13 Threats to validity

- **Internal:** the observed proxy is not semantic gold; leakage control (allow-intent-path-leakage off, parent-only inputs) is mandatory; no hidden-test peeking.
- **External:** n and single-repository generalization; V2 and Saleor/NestJS are the mitigation, with per-repository primary metrics.
- **Statistical:** low negatives in the development label; multiple comparisons; wide random bands — addressed by pre-registration, class-balance gates, and bootstrap uncertainty.
- **Representation:** LLM serialization cost spans providers; we report both recorded and frozen-pricing cost and keep provider pinned as far as allowed.

14 Expected contributions

1. Sparse explicit impact-plan representation with controlled evidence.
2. A reproducible real-commit impact-localization dataset and protocol (repository, eligibility, split, proxy, hashes).
3. Fair cheap/classical/LLM/agent comparison under common file-level metrics and explicit budgets.
4. A bounded/selective-compute verification protocol (Route A/B) if supported by evidence.
5. Cross-repository correctness–cost evidence.
6. Reproducibility artifacts and leakage-control methodology.

No positive algorithmic result is promised.

15 Work plan / timeline

Window	Activity
2026-09/10	Proposal V1; dataset V2-LARGE freeze + build
2026-10	V2 development inference; classical + cheap baselines; Route B design
2026-11	Bounded verification experiments; Saleor protocol + inference
2026-12	NestJS cross-ecosystem check; Pareto analysis
2027-01/02	Analysis, replication, threat revisions
2027-03/04	Write-up; internal test confirmation
2027-05	Submission-ready thesis

16 References

The authoritative bibliography is `references.bib`. Verified primary sources include (venue where confirmed): Agentless (arXiv, 2024); CodePlan (ICSE 2024); RepoCoder (EMNLP 2023); AutoCodeRover (ISSTA 2024); RepoGraph (ICLR 2025); LocAgent (arXiv 2025, upstream pin evaluated under a shared protocol); the Shichao Zhang adaptive-kNN/cost-sensitive line (TKDE 2021/2022; Neurocomputing 2020); GraphLocator (FSE 2026, LLM causal-issue-graph localization); and the classical change-impact/ripple-effect line. Preprints are marked as such in the bibliography.